

MScheduler: Leveraging Spot Instances for High-Performance Reservoir Simulation in the Cloud

Felipe A. Portella^{*†‡}, Paulo J. B. Estrela^{*}, Renzo Q. Malini^{*}, Luan Teylo^{§¶},
Josep L. Berral^{‡†}, Lúcia M. A. Drummond[¶]

^{*}Petróleo Brasileiro S.A. (PETROBRAS), Rio de Janeiro, Brazil

[†]Barcelona Supercomputing Center (BSC), Barcelona, Spain

[‡]Universitat Politècnica de Catalunya (UPC), Barcelona, Spain

[§]Institut National de Recherche en Informatique et en Automatique (Inria), Bordeaux, France

[¶]Universidade Federal Fluminense (UFF), Rio de Janeiro, Brazil

{felipeportella, paulo.estrela, renzo}@petrobras.com.br, luan.teylo@inria.fr, josep.ll.berral@upc.edu, lucia@ic.uff.br

Abstract—Petroleum reservoir simulation uses computer models to predict fluid flow in porous media, aiding to forecast oil production. Engineers execute numerous simulations with different geological realizations to refine the accuracy of the model. These experiments require considerable computational resources, which are not always available within the on-premises infrastructure. Commercial public cloud platforms can offer many advantages, such as virtually unlimited scalability and pay-per-use pricing. This paper introduces MSCHEDULER, a meta scheduler framework for reservoir simulations at Petrobras, a Brazilian energy company. It efficiently executes jobs in the cloud, utilizing spot Virtual Machines (VMs) to reduce costs and ensure job completion even with VM termination. Contributions include a novel methodology for reservoir simulation checkpointing, a cost-based scheduler, and an analysis of the strategy using real production jobs from Petrobras.

Index Terms—Cloud Computing, Spot Instances, Reservoir Simulation

I. INTRODUCTION

Petroleum reservoir simulation is a subset of reservoir engineering that employs sophisticated computer models to predict fluid flow (typically oil, water, and gas) through porous media [1]. Engineers can make decisions throughout these simulations and effectively forecast future oil production. This process involves designing models of oil fields and performing computationally demanding calculations to guide extraction strategies. The reservoir models do not provide value to the business if they do not accurately represent the real reservoirs. Usually, to refine the model's accuracy, engineers employ a technique known as History Matching (HM). The primary objective of HM is to assign optimal values to the model parameters such that the mathematical representation of the reservoir replicates the behavior observed during the production period. This leads to improved forecasting abilities and reduced uncertainty. However, the HM workflow can be computationally demanding, often requiring numerous simulations of the same reservoir model with different geological

realizations. Therefore, in practice, engineers launch hundreds of simulations as individual jobs to process slightly different problems in parallel.

Additionally, as reservoir simulation models continue to increase in size and complexity, and with unpredictable demands, on-premises capacity may not be sufficient to meet processing requirements at times. Setting up new infrastructures for large projects is seldom straightforward, as it can be costly and may result in investments being tied up in idle on-premises computing once the project is completed. On the contrary, cloud platforms offer almost limitless computing resources and storage space that users can rent as required, without the need for equipment ownership and maintenance. In this context, an interesting strategy is the adoption of a hybrid cloud approach that combines the on-premises infrastructure with public cloud resources, allowing scaling up on demand. Most cloud platforms allow users to dynamically acquire computational resources wrapped as Virtual Machines (VMs). Users can deploy VMs based on their application requirements, such as CPU, memory, and I/O, in a pay-as-you-use price model. However, there is a trade-off between performance and budget: the greater the demand for resources and computing power, the higher the associated cost for users.

Cloud providers, such as Amazon Web Servers (AWS), offer a variety of VMs in different contract models, with varying guarantees in terms of pricing, contractual commitment, and availability. For instance, Amazon Elastic Compute Cloud (EC2) has three main contract models, also called markets: reserved, on-demand, and spot. All three markets offer a wide range of VMs types to suit different user requirements. According to AWS, instance types differ in the combinations of virtual CPUs (vCPUs), memory, storage, and networking capacity, allowing users to choose the appropriate mix of resources for their applications. AWS groups instance types into families according to their use case [2].

In the spot market, the availability of VMs fluctuates

according to the cloud's current demand. If there are not enough resources to meet the client's requests for on-demand VMs, the cloud provider may interrupt the spot VMs – VM termination – either temporarily or definitively. Despite the risk of unavailability, the main advantage of spot VMs is their monetary cost. Spot instances are often significantly cheaper than on-demand VMs with the same computational capabilities, with discounts that can reach 90% [3]. This cost-effectiveness is directly derived from the spot model: it represents a strategy for cloud providers to monetize their idle resources by offering them at substantial discounts. To handle VM termination in such scenarios, the use of fault tolerance techniques becomes crucial. These techniques are designed to ensure that applications reach completion, even in cases of failure (or unexpected VM termination). Checkpointing and Restarting (CR) is one of the most widely used fault tolerance techniques for long-running applications. With CR, application states are periodically saved to a stable, fail-free storage resource. Thus, in the event of failure, the application can be restarted from the last recorded state on a different machine (or VM).

This paper presents MSCHEDULER, a meta scheduler framework for reservoir simulations that serves as an intermediary layer between the client and the cloud, at Petrobras, a state-owned Brazilian energy company. The framework efficiently sends jobs to the cloud and takes advantage of spot VMs to reduce costs. It manages the entire execution of the application, including coordinating checkpoint records and ensuring job completion, even in the event of VM termination.

MSCHEDULER explores the utilization of spot instances provided by Amazon Web Services (AWS). We particularly focus on AWS Elastic Compute Cloud (EC2) instances for handling these simulator workloads. For this current work, AWS was chosen as it is one of the cloud providers with which Petrobras has contracts. However, the proposed framework can be extended to other cloud providers with minor adjustments. Furthermore, due to the Brazilian government's requirement for data sovereignty, a copy of the data must be maintained within Brazilian territory. Therefore, a base scenario to fulfill this requirement was to have the storage within Petrobras, a solution that was successfully implemented when working with EC2 instances in the AWS São Paulo region (SA-EAST-1) due to the high-speed network interconnection of Petrobras and the Internet exchange point where AWS is present (via AWS Direct Connect). As we consider leveraging the availability and lower spot costs of different AWS regions globally, we must consider the time and cost penalties associated with the data transfer between cloud regions, as it is not feasible to exclusively store the simulation models abroad in compliance with the data sovereignty restriction.

Thus, the main contributions of this work are as follows:

- The development of a novel methodology for reservoir simulation checkpointing and recovering, capable of dealing with spot revocation in an efficient manner;
- The creation of a cost-based scheduler framework for effectively managing the application life-cycle on spot

or on-demand instances, considering the requirements on data placement (local-global);

- The analysis of the proposed strategy in a real Cloud provider (i.e. AWS), using real production jobs from Petrobras as principal benchmarking and target workload.

This paper is organized as follows: Section II provides an overview of the state of art in cloud-based reservoir simulation workloads and spot instances. In Section III, we detail our proposed strategy, framework architecture, and methodology. Section IV presents experimental results and analyses in a real cloud provider environment. Finally, Section V summarizes the key findings and discusses implications and future research directions.

II. RELATED WORK

To the best of our knowledge, only a limited number of papers have addressed reservoir simulations on cloud platforms, and none of them have specifically delved into the exploration of spot instances, as presented next.

Eldred et al. conducted one of the pioneering works [1] in exploring the application of High Performance Cloud Computing (HPCC) in the upstream oil and gas industry. The authors carried out a pilot project using Amazon Web Services (AWS) as a public HPCC platform using Eclipse, the industry standard reservoir simulation software. The project aimed to demonstrate the benefits of HPCC, such as flexibility, accessibility, and cost reduction, particularly for small to midsize companies that may not wish to invest heavily in on-premises infrastructure. The paper presents a framework that incorporates a simplified break-even point cost model, which was based on the HPC resource utilization period. The deployment of the VMs was automated using scripts through AWS's Command Line Interface (CLI), and the cloud model was based on fixed cost. Although specific details were not provided, it is likely that it was based on on-demand pricing.

Flister et al. presents in [4] an orchestration tool used to automate the deployment of an HPC cluster on Microsoft Azure to run the reservoir simulators tNavigator and Eclipse. The chosen tool, CycleCloud, provides comprehensive features to dynamically scale resources in response to workload demands, which greatly aids in optimizing resource utilization and cost efficiency. CycleCloud also supports various job schedulers and in that study the researchers decided to deploy an IBM LSF cluster with job queues based on various machine types, thus enabling differentiation in terms of memory capacity and CPU power. The performance and cost management aspects of [4] are based on the end user's manual selection of the appropriate queue. Although the researchers included monitoring dashboards with a monthly breakdown, they did not automate the logic of submission to different queues, as we have proposed in our study, nor did they mention spot instances or incorporating checkpoints.

Noor et al., in [5], reviewed the benefits of running reservoir simulations in a cloud environment and demonstrated efficiency and cost savings. They argued that a cloud-based reservoir simulation software as a service model simplifies

the process and reduces the costs associated with hardware acquisition and maintenance. Additionally, they also presents a workflow that uses cloud-based simulation services with containerization, Kubernetes optimization, and integrated data analysis tools to achieve efficient uncertainty analysis and history matching in reservoir simulations.

Finally, Portella and Souza, in [6], give a brief overview of the reservoir engineering area and its reservoir simulation applications. They also describe a project to move part of the daily reservoir simulation demand of a globally recognized Brazilian energy company to third-party cloud providers. This project also addressed interesting challenges regarding Network Attached Storage (NAS) synchronization issues and how to balance on-premises and cloud provisioning.

III. PROPOSED STRATEGY

Computer Modelling Group's (CMG) reservoir simulators are renowned in the oil and gas industry for their exceptional ability to model and analyze complex reservoir systems [7]. Among the extensive CMG product family, GEM and IMEX reservoir simulators are the most used tools in Petrobras for their ability to represent compositional and black-oil reservoir systems, respectively. These simulators are designed to explore the power of high-performance computing (HPC) systems efficiently using parallel processing techniques.

These CMG simulators have a special feature called RESTART that is explored in the context of this work as a checkpoint mechanism. This feature is designed to write to a special file the state of the indicated time steps and save execution time at a frequency chosen by the user in specific studies, such as sensitivity analysis or history matching, where all the simulations share a common history period. To generate a restart file, we can use a UNIX signal to interrupt the simulation and perform certain actions, such as creating the file and deciding whether to continue or quit. We will employ this file to establish a checkpoint towards resuming the simulation in case of spot instance revocation.

In this context, to effectively manage the execution of reservoir simulations in the cloud and explore the spot market, we propose MSCHEDULER, a scheduling framework that handles the entire application execution cycle while utilizing spot VMs to reduce costs. MSCHEDULER comprises two main components: the LAUNCHER and the EXECUTION MANAGER. The LAUNCHER acts as the user interface for application submission, receiving simulation parameters, and submitting the simulation to the cluster resource manager (Slurm). Furthermore, the launcher determines the market, region and VM that will be used to run the simulation. Each simulation is accompanied by an EXECUTION MANAGER: a daemon that actively monitors the states of the VMs and promptly responds to VM terminations, particularly when spot VMs are employed. Moreover, that daemon is responsible for managing the checkpointing and recovery of the applications.

Figure 1 illustrates the conceptual architecture of MSCHEDULER, highlighting its two main components (shown in red).

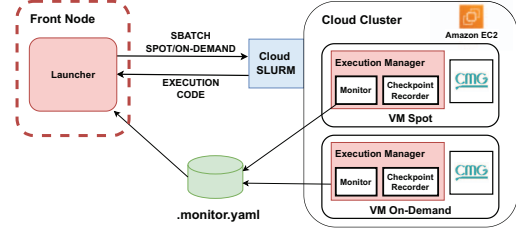


Fig. 1. Conceptual architecture of MSCHEDULER, including the two main components (in red) and some procedures (in white). The figure illustrates the execution cycle of a simulation submitted throughout the framework.

To explain the utilization of MSCHEDULER and the components involved in execution, let us consider a typical scenario: a user submits a simulation to be executed using the LAUNCHER on the front node. The LAUNCHER then decides to execute the simulation in a spot VM and sends it to Slurm using the SBATCH command. Slurm allocates the VM in EC2 and dispatches the simulation. However, at some point, the provider may terminate the spot VM. To ensure uninterrupted execution, the LAUNCHER takes action and re-submits the simulation to another VM, such as an on-demand VM. During this process, the simulation is restored from a previously recorded checkpoint that was saved by the EXECUTION MANAGER. Next, we will explain each component involved in these steps.

A. LAUNCHER

To initiate a simulation, the user interacts directly with MSCHEDULER's LAUNCHER through a command-line interface. The user provides three main parameters: the batch file to be submitted to Slurm, the directory where the simulation files and MSCHEDULER files will be stored (referred to as *workdir*), and an estimated runtime (MAKESPAN) for the simulation. The MAKESPAN represents an estimate of the simulation execution time in an on-demand VM that is used as a reference (vm_{on}^{id}). Currently, we assume that users are sufficiently familiar with the simulation to provide this estimation. This assumption is reasonable in the given scenario, as engineers in oil companies regularly perform reservoir simulations, often on a weekly or even daily basis.

Once the user parameters are received, the LAUNCHER executes a selection procedure to determine the VM that will be used to run the simulation. For this study, we select an on-demand VM from the region closest to the used storage, denoted as S , as the standard option referred to as vm_{on}^{id} . To optimize cost efficiency, we propose a VM selection strategy aiming to find a spot or on-demand instance with performance equivalent to vm_{on}^{id} for each EC2 region. This strategy prioritizes minimizing the monetary cost of execution while considering the potential time implications of writing checkpoints from a region distant from the storage location. Algorithm 1 presents the proposed strategy, and the notation used in the algorithm is detailed in Table I.

Algorithm 1 takes three parameters as input: the estimated makespan provided by the user (MAKESPAN), the checkpointing interval (INTERVAL), and the average time to record a **single** checkpoint of the simulation (*dump*). Both the INTER-

TABLE I
NOTATION USED IN THE INSTANCE SELECTION ALGORITHM.

Name	Description
OVERHEAD_R^S	Overhead of using storage in region S while executing the simulation in a VM in region R .
MAKESPAN	Estimated runtime of the simulator in vm_{on}^{id} .
INTERVAL	Time interval between consecutive checkpoints.
$dump$	Average checkpoint recording time.
vm_{best}	VM with the minimum cost for executing the simulator.
vm_{on}^{id}	Standard option VM in the on-demand market.
$cost(vm_i)$	Monetary cost of vm_i .
SETVM_{id}	Set of available Spot and On-demand VMs.
Q	Quarantine time during which a recently revoked VM should be excluded from the SETVM_{id} list.

VAL and the $dump$ are defined on a pre-experiment basis and described in Section IV. Additionally, the algorithm considers the time OVERHEAD_R^S for executing the simulation in a region R that may be far from the region where the data is stored (S). This overhead is predefined and empirically estimated from the distinct regions evaluated in this work (Section IV-A).

Furthermore, it is necessary to determine the list of VMs that will be considered for selection (SETVM_{id}). Defining this list is a significant task on its own, as it directly impacts the algorithm's efficiency in searching for suitable VMs. It would require a comprehensive study of the performance of different VMs specifically tailored to the simulations (similar to the one conducted in Teylo *et al.* [8] for another type of application). Therefore, for the purposes of the discussions presented in this paper, we will limit our study to one type of VM, the c5.24xlarge, and two regions: US-EAST-1 and SA-EAST-1. Thus, SETVM_{id} is made up of four VMs: two c5.24xlarge VMs for each region, one in the spot market and the other in the on-demand market.

Algorithm 1 begins by initializing vm_{best} as the standard on-demand VM instance vm_{on}^{id} and defines MIN_{cost} as the monetary cost of executing the simulation using vm_{on}^{id} , taking into account the estimated MAKESPAN. It then iterates through the set of available VMs (spot and on-demand) in the list SETVM_{id} . For each VM in the list, it checks if it is a spot VM. If it is, the algorithm computes the additional time overhead caused by the checkpointing procedure executed for spot VMs. To calculate this overhead, the algorithm first obtains the region of vm_i (Line 6). It then calculates the number of checkpoints (n_{ckp}) required based on the INTERVAL and the MAKESPAN. After determining the number of checkpoints, the algorithm estimates the checkpoint time ($time_{ckp}$) in Line 8. This estimate takes into account both the $dump$ and OVERHEAD_R^S . If $S = R$, the OVERHEAD_R^S will be 1, indicating no overhead since the VM and the storage are in the same region. However, if OVERHEAD_R^S is greater than 1, indicating that the VM is in a different region, the overhead is considered in the estimated makespan ($\text{MAKESPAN}'$). Subsequently, the algorithm compares the execution cost of the current VM in the set, considering the $\text{MAKESPAN}'$. If a lower cost is found, the vm_{best} and MIN_{cost} variables are updated accordingly. Finally, the algorithm returns the selected VM instance at the lowest cost. This procedure ensures that the chosen VM

minimizes the overall cost of execution.

Algorithm 1 Instance Selection

Input: MAKESPAN, INTERVAL, $dump$

```

 $vm_{best} \leftarrow vm_{on}^{id}$ 
 $\text{MIN}_{cost} \leftarrow cost(vm_{on}^{id}) \times \text{MAKESPAN}$ 
for  $vm_i \in \text{SETVM}_{id}$  do
     $\text{MAKESPAN}' \leftarrow \text{MAKESPAN}$ 
    if  $vm_i$  is Spot then
         $R \leftarrow region(vm_i)$ 
         $n_{ckp} \leftarrow \lfloor \text{MAKESPAN} / \text{INTERVAL} \rfloor$ 
         $time_{ckp} \leftarrow n_{ckp} \times (dump \times \text{OVERHEAD}_R^S)$ 
         $\text{MAKESPAN}' \leftarrow time_{ckp} + \text{MAKESPAN}$ 
    end if
    if  $\text{MIN}_{cost} > (cost(vm_i) \times \text{MAKESPAN}')$  then
         $vm_{best} \leftarrow vm_i$ 
         $\text{MIN}_{cost} \leftarrow cost(vm_i) \times \text{MAKESPAN}'$ 
    end if
end for
return  $vm_{best}$ 

```

As can be seen in Algorithm 1, our selection strategy does not incorporate the Amazon Placement Score, a recently introduced feature by AWS. This score is capable of recommending an AWS region (or availability zone) based on Spot requirements [9]. According to Amazon, the Placement Score indicates the likelihood of a requested spot instance being successfully allocated in a region, with a score of 1 meaning that the instance is unlikely to be allocated and a score of 10 being the highest. During this study, we conducted tests that unveiled a highly subjective behavior associated with the current score. Spot instances were successfully allocated in regions with very low scores, whereas in regions with the same score, the allocation was denied. Since this feature is new and lacks comprehensive studies, we have decided not to consider it in the present version of our framework. However, it will be taken into account in future work.

As mentioned previously, when a spot VM is selected, it is susceptible to revocation by the cloud provider. Consequently, in the event of spot termination, the Instance Selection algorithm is executed once again to select the next VM that will be used to restart the application. However, that selection needs to consider the following: if a spot VM allocated in the region R was just terminated by the provider, all VMs of the same type in this region are currently facing a high probability of termination. Therefore, even if we could immediately request the same VM in R , we should avoid doing so, as it is likely to be terminated again. To address this issue and avoid allocating the same VM after its termination, we implement a simple quarantine routine. This routine keeps vm_i out of the SETVM_{id} list for a period of Q minutes. Thus, whenever the provider terminates a spot vm_i , it is immediately removed from SETVM_{id} (before the execution of Algorithm 1). After the Q minutes elapse, vm_i is placed back on the list.

B. EXECUTION MANAGER

When a simulation is initiated in a VM, the framework also dispatches the EXECUTION MANAGER, a daemon that runs within the VM and coordinates the checkpointing and restart procedures. Additionally, the EXECUTION MANAGER is responsible for detecting if a spot VM is going to terminate by intercepting the interruption notices in the VM's metadata [10].

The EXECUTION MANAGER is a state-driven procedure that takes actions based on the simulation's states: i) *starting*, when the simulation is being initiated; and ii) *running*, when the simulation is in progress.

a) *Starting*: Before starting the simulation, the manager checks if the current simulation needs to be restarted from a checkpoint. To do this, the EXECUTION MANAGER verifies the presence of the *.monitor.yaml* file in the *workdir*, used to keep track of the simulation states and contains information such as the current state of the simulation and the execution ID. If it exists, the manager recovers the information and restarts the simulation from the latest checkpoint. Otherwise, the simulation starts from the beginning. When the simulation starts, the manager updates the simulation ID and its state (running) in the *.yaml* file. Additionally, if the simulation is running in a spot VM, the manager initiates the checkpoint recorder (for more details, see Section III-C).

b) *Running*: During the execution of the simulation on a spot VM, the EXECUTION MANAGER continuously monitors the VM's metadata to intercept interruption notices, provided by AWS two minutes before the instance is interrupted. Therefore, upon receiving the interruption notice, the manager has a two-minute window before the VM is interrupted. During this time, the EXECUTION MANAGER initiates a checkpoint recording with the aim of creating a final checkpoint. The purpose is to preserve most of the work done on the spot VM and minimize potential loss.

It is important to note that there is no guarantee that this last checkpoint will be successfully recorded in the given two minutes. Consequently, relying solely on it is insufficient to guarantee that the work done on the spot VM will be saved, posing the risk of losing everything. To mitigate this risk, we have implemented a mechanism to record checkpoints periodically (more details in Section III-C). In summary, our goal is to save nearly all of the work performed. However, if this is not feasible, we can still minimize the loss by leveraging periodic checkpoint recordings throughout the execution. Note that if the simulation is on an on-demand VM, the EXECUTION MANAGER does not need to monitor for terminations or failures. In such cases, the manager waits until the end of the simulation without the need for additional monitoring.

C. Checkpoint Recording and Rollback Recovery

When the simulation runs on a spot VM, the EXECUTION MANAGER initiates the checkpoint recorder, which is a procedure that runs in a separate thread and coordinates the checkpointing process of the simulation. The checkpoint recorder receives the *INTERVAL* as input and performs the necessary actions to create a checkpoint of the simulation according to this interval. These actions include: i) Creating a *.signal* file in the *workdir*; ii) Waiting for the checkpoint recording procedure; and iii) Copying the checkpoint files.

It should be noted that, as mentioned earlier, this work explores the existing checkpointing and recovery capabilities of the reservoir simulators that we are considering. As demonstrated in Section IV-A, the checkpoint mechanisms in these

simulators are highly efficient in terms of time. However, a few additional steps are necessary to utilize these mechanisms in the context of spot VMs. In these simulators, a checkpoint can be recorded by adding a *.signal* file in the *workdir* with a specific numerical code written in it. At each time step, the simulator checks the *workdir*, and if a *.signal* file is found, it executes the procedure in accordance with the numerical code. In our case, we use code 2 to record a checkpoint and continue the simulation. Therefore, the first step performed by the checkpoint recorder is to periodically write the *.signal* file to the *workdir*, triggering the checkpointing process.

The second step implemented in the recorder is the waiting operation. When the simulator is writing a checkpoint, we need to ensure that the recorder will wait until the entire process is completed. However, the simulator does not indicate that the checkpointing procedure is in progress. To address this issue, we implemented a waiting procedure that checks the checkpoint files written by the simulator (*.sr3* files). We do this by retrieving the file's last modified time and checking if it has changed in the last second. If that is not the case, we then know that the checkpoint procedure has ended. This procedure is very efficient and does not incur an overhead to the checkpointing process.

Finally, the last additional step is the copy procedure. After the simulator writes the checkpoint file, we need to make a copy to keep track of the checkpoints. This is necessary because the simulator overwrites the checkpoint file whenever a new checkpoint is created. In this case, we implemented a round-robin mechanism that keeps the last three generated checkpoint files. The first generated file is renamed as "checkpoint 1," the second as "checkpoint 2," and so on. Once the three initial files are generated, we start overwriting them, retaining the newest ones. As we demonstrate in Section IV-A, the copy operation represents the highest overhead among all operations involved in the checkpointing procedure.

In addition to recording the checkpoints, this procedure is also used for the recovery of the simulation when the execution is moved to another VM due to spot termination. Once again, we utilize the already implemented (and efficient) mechanisms of the simulators, which allow us to simply pass a parameter indicating the checkpoint file that needs to be read to recover the application. Thus, the recovery procedure performed by the recorder module first retrieves all the checkpoint files in the *workdir*, sorts them by the newest one, and passes the file to the simulator command line. In the event of an error, such as a corrupted checkpoint file from which the simulation cannot restart, the second checkpoint file in the list is sent to the simulation, and so on.

IV. EXPERIMENTAL RESULTS

We evaluate the proposed strategy based on two metrics: execution time and monetary cost. First, we conduct a study to assess the impact of the checkpoint interval on both the overall checkpoint procedure time and the simulation time. After that, we explore various failure scenarios and evaluate the proposed framework using two simulation models. Please

note, **all tests** in this paper were conducted using actual AWS cloud resources.

For all tests in this section we used as workload a semi-synthetic reservoir simulation model, which will be referred to as Pre-salt 100. This is a semi-synthetic representative workload of the complex pre-salt reservoirs in offshore Brazil explored by Petrobras. Although compositional fluid modeling is the most common, we chose the black-oil version for the IMEX simulator because it is faster. To save time and budget, we created a version that, instead of running the simulation for 50 years, runs for *just* 20 years. Throughout the remainder of this paper, we will refer to the models as PRE-SALT and SHORT-PRE-SALT, respectively. All IMEX simulations were performed with PARASOL solver using 40 threads.

A. Evaluation of Checkpoint Overhead

To assess the impact of application checkpointing on long-running jobs, we examined two checkpoint intervals: 5 minutes and 30 minutes. All experiments were carried out with the storage and VMs in the same geographical region: our environment uses an on-premise NAS (Network Attached Storage) located in Brazil, allowing file sharing through the NFS (Network File System) protocol, and the VM was allocated in the SA-EAST-1 region. As a baseline comparison, we also executed the model without checkpointing (baseline execution). All tests were repeated three times. Table II summarizes the execution information.

TABLE II
EVALUATION OF CHECKPOINT INTERVALS. THE TABLE DISPLAYS THE AVERAGE VALUES OF 3 EXECUTIONS.

Checkpoint Interval	Nº of Checkpoints	Checkpoint Duration(s) ¹	Copy Duration(s)
5 minutes	162.33	12.54	153.91
30 minutes	17.66	13.49	22.84

¹ This column represents the average time from when the “.signal” file is written until the simulator finishes writing the checkpoint file.

As shown in Table II, in the checkpoint duration column, the checkpoint interval does not significantly impact the time required for recording the application checkpoint. However, it affects the time taken to copy the checkpoint files. When comparing a 5-minute interval to a 30-minute interval between checkpoints, there is an average reduction in time of approximately 85.1% for the copy process. The impact of this overhead on the total execution time of the simulation can be observed in Figure 2. As depicted, the total simulation time increased by over 63% with a 5-minute interval compared to the baseline case (without checkpoints), while an increment of less than 1% was observed with a 30-minute interval. This discrepancy occurs due to the high frequency of checkpoints at the 5-minute interval, causing the simulation to pause frequently for checkpointing recording. Consequently, the overhead of pausing and restarting all 40 threads becomes prohibitive, significantly impacting the overall execution time.

Another crucial aspect of our framework is the exploration of distinct AWS regions to find the most cost-effective VM option. As mentioned previously, this study considers two geographically distant regions: Brazil and the US. Consequently,

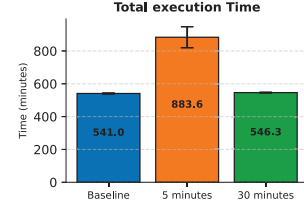


Fig. 2. Bar chart presenting the average execution time of the entire simulation for three distinct cases: baseline (without checkpointing), 5-minute interval, and 30-minute interval. Each test was repeated three times.

one potential impact of this solution is the overhead associated with I/O when saving the checkpoint files. Since our NAS is located in Brazil, this impact is particularly noticeable when the VMs are located in the US region. To evaluate this impact, we conducted a test using the PRE-SALT model with a 30-minute checkpoint interval. The results are shown in Figure 3.

As expected, the execution time in the US region increases compared to the execution in Brazil. This increase is due to the overhead added by the transfer time over NFS, which is influenced by the distance between the regions. We observed an 11% increment in execution time when comparing the two regions. Furthermore, when analyzing the checkpoint overhead, it can be observed that the average time to record and copy the checkpoint files (*dump*) is approximately 36.33s when both the storage and VM are allocated in the same region. However, when they are in distinct regions, this time increases to approximately 166.45s. This represents a 4.58x increment in the checkpointing time.

These results reinforce our strategy, considering the time overhead when selecting the VMs. In other words, we need to take into account not only the price of the VM but also the increase in time (and consequently the total monetary cost) when deciding in which region the simulation will be executed.

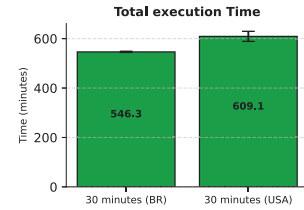


Fig. 3. Comparison of Total Execution Time of Simulations in Brazil (SA-EAST-1) and US (US-EAST-1).

B. Failure Analysis

In the following tests, our objective is to evaluate the performance of our solution across various execution scenarios, focusing on different termination patterns of spot VMs. We aim to assess the MSCHEDULER in terms of monetary cost and execution time when spot VMs are subjected to revocation. To replicate real-world conditions, we emulate the termination procedure executed by AWS when it determines that a running spot VM will be terminated. Therefore, in these tests, we not only replicate the VM revocation but also mimic the notification process.

a) *Emulating Spot Revocation:* For the purpose of emulating spot revocation, we have developed a procedure called TERMINATION INJECTOR. This procedure is initiated concurrently with the spot VM and is responsible for the following

tasks: 1) Determining when the spot VM will be terminated; 2) Updating the VM's metadata with the termination notification; and 3) Terminating the VM. To model the VM termination time, we employed a Poisson distribution. The Poisson distribution is a discrete probability distribution that describes the number of events occurring within a fixed interval of time or space. It is commonly used to model events that occur randomly in time, making it suitable for simulating the termination of a spot VM [8], [11], [12].

In the TERMINATION INJECTOR, the termination rate (λ) represents the average number of spot interruptions per hour. Therefore, the expected number of spot interruptions can be calculated as $\frac{1}{\lambda}$. Hence, to generate the termination time of a spot VM, denoted as τ , we obtain a random sample from the Poisson distribution and multiply it by 3,600. In other words, τ represents how many seconds the VM will be available before its termination.

Once the termination time of the spot VM is determined, the TERMINATION INJECTOR keeps track of the duration of the VM allocation. After $\tau - 120$ seconds, the procedure generates the termination notification that is captured by the EXECUTION MANAGER. As explained in Section III-B, AWS sends a notice of spot termination via the VM metadata. To emulate that notification, we use Amazon EC2 Metadata Mock (AEMM), a tool recommended by AWS to emulate VM metadata [10]. The AEMM receives $\tau - 120$ as input, that is, the number of seconds when the termination notification needs to be written in the metadata. However, the tool does not terminate a VM; it only emulates the updates in the metadata that would happen in a real execution case. Therefore, after two minutes of the notification, the TERMINATION INJECTOR executes the *poweroff* command inside the VM.

b) *Experimental Setup*: Table III provides the characteristics of each model evaluated, including the execution time without a checkpoint (MAKESPAN) and the average time required for the checkpointing process (*dump*). Note that Table III values are an average of three executions. Moreover, the SHORT-PRE-SALT model despite being an altered version of the PRE-SALT model does not change the characteristics of the model, only its execution time. Therefore, the *dump* of the Short version is the same as that of the full model.

For the evaluation, we considered three termination rates (λ): 0.1, 0.5, and 1.0. Therefore, when we increase the λ , the probability of terminating a spot VM soon in execution increases, with the termination rate equal to 1 representing execution cases with a high probability of terminating the VM within the first hour of execution.

To ensure complete control over the termination process and avoid any potential disruptions in our results, all tests in this section were performed using on-demand VMs. This approach was chosen to prevent the possibility of AWS terminating spot VMs during the tests. To execute the tests using on-demand VMs, we made adjustments to the framework to ensure that it continued to make decisions based on spot and on-demand prices. The framework allocated VMs in the usual manner, but behind the scenes, we redirected the simulations to on-demand

TABLE III
SIMULATION MODELS AND PARAMETERS CONSIDERED IN THE TESTS.
THE TIMES WERE ESTIMATED USING THE VM c5.24XLARGE AND SERVED
AS INPUT TO ALGORITHM 1.

Model	MAKESPAN (s)	<i>dump</i> (s)
PRE-SALT	27,340	36.37
SHORT-PRE-SALT	8,644	36.36

VMs. Additionally, we took care to ensure that the on-demand VMs were allocated in the same region as determined by the framework. By doing so, we maintained the integrity of the framework's decisions.

For the overhead value, we refer to the results presented in Section IV-A, resulting in $\text{OVERHEAD}_{US}^{BR} = 4.58$. Once again, we used the c5.24xlarge VM in both the spot and on-demand markets for our evaluations. The reference VM, vm_{on}^{id} , was the on-demand VM in the SA-EAST-1 region. For the quarantine period and checkpoint interval, we used 30 minutes ($Q = 1800$ s and $INTERVAL = 1800$ s).

c) *Results*: Figures 4 and 5 present the results of this evaluation. As can be seen, the MSCHEDULER successfully reduced the execution costs¹ for **all** execution cases, even in the scenario where failures are likely to occur within the first hour of execution ($\lambda = 1.0$). These results demonstrate the cost-effectiveness of our selection method: selecting the cheapest VM while considering the time overheads caused by the use of distinct regions and checkpoints (in the case of spot VMs) proves to be an effective strategy when forwarding long-running jobs to the cloud. On average, we were able to save up to 65.99% of the monetary costs in cases with a low probability of spot revocations ($\lambda = 0.1$) and up to 40.05% in the worst-case scenario ($\lambda = 1.0$) for the model SHORT-PRE-SALT. For the PRE-SALT model, the monetary reductions were 66.23%, 43.80%, and 32.18% for λ values of 0.1, 0.5, and 1.0, respectively.

We observed an increase of up to 20% in execution time across all scenarios. Specifically, for PRE-SALT, the average makespan increased by 19.39%, 20.82%, and 8.26% with λ values of 0.1, 0.5, and 1.0, respectively. Similarly, for SHORT-PRE-SALT, the increases were 20.41%, 26.91%, and 12.85%. During the tests, we noticed that the LAUNCHER tended to select spot VMs located farthest from the storage (US-EAST-1 region), while the on-demand VMs were chosen closest to the storage. As a consequence, the makespan faced a more significant increase for cases with failure rates of 0.1 and 0.2 compared to those with a high failure rate $\lambda = 1.0$. This can be attributed to the fact that for $\lambda = 0.1$ or 0.5, the simulation ran for a longer duration in the distant region since the spot VMs were available for more time, which increased the makespan due to I/O-related overheads.

The increase in time was expected for scenarios with spot terminations, as they introduce two additional overheads: the recovery time and the time required to launch the next VM used to restart the simulation. Therefore, the makespan in-

¹All the monetary costs reported in this section were derived from executions from July 1st to July 27, 2023, reflecting the corresponding prices of the VMs during that period.

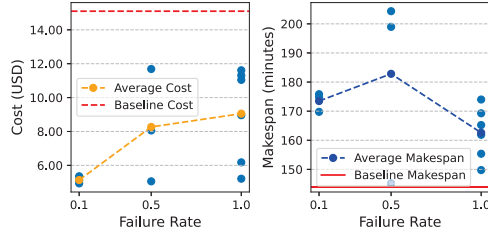


Fig. 4. Monetary cost and makespan of multiple executions of the SHORT-PRE-SALT model, considering various rates of spot VMs revocations. Each blue dot represents an individual execution.

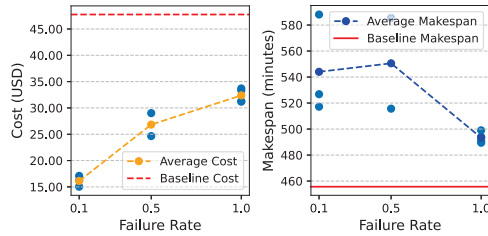


Fig. 5. Monetary cost and makespan of multiple executions of the PRE-SALT model considering various rates of spot VMs revocations. Each blue dot represents an individual execution.

creases with the number of terminations. This can be observed in the outlier values present in the makespan graph of the PRE-SALT model for $\lambda = 0.5$ (Figure 5) and in the outlier value present in the makespan graph of the SHORT-PRE-SALT model (also for $\lambda = 0.5$). In these cases, the simulations experienced 4 revocations.

The outliers in Figures 4 and 5 also illustrate the impact of quarantine time on our solution. For some of the tests with $\lambda = 0.5$, we observed the following effect: the LAUNCHER selected the first spot VM, and after a while, the provider terminated that VM. The LAUNCHER then placed it in quarantine and selected the second spot VM. Since the failure rate was not very high, the second spot VM was terminated after more than 30 minutes of execution. Therefore, when it was terminated, the first spot VM was already out of quarantine, resulting in its selection again. This behavior resulted in a ping-pong effect in which the simulation jumped from spot to spot, increasing the makespan. Note that we did not observe the same pattern for $\lambda = 1.0$ as terminations are likely to occur before quarantine time, moving the simulation to an on-demand VM sooner.

These results demonstrated the effectiveness of our solution in saving monetary costs (the main objective of this work) and emphasized the need for careful selection of parameters in the VM Selection Algorithm.

V. CONCLUSIONS AND FUTURE WORK

This work proposes MSCHEDULER, a scheduler framework tailored for the execution of reservoir simulations in the cloud that manages the entire execution cycle of these applications. The developed framework takes advantage of spot virtual machines, including a fault-tolerant mechanism to treat eventual revocations.

Although some companies (e.g., CMG Cloud [7]) offer a solution to execute reservoir simulations in the cloud, the VM selection of the proposed framework has the advantage of

analyzing the prices of spot and on-demand VMs for each considered EC2 region, thereby optimizing cost efficiency. This strategy prioritizes minimizing the monetary cost of execution while considering the potential time implications of the VMs accessing data from a region far from the storage location. Our results in real cloud environments demonstrate the effectiveness of MSCHEDULER: for the PRE-SALT model, even in cases where spot VMs have a high failure rate, it reduced the average monetary cost by up to 33.88%, with only an 11.18% increase in makespan compared to executions using on-demand VMs. In the best case, the monetary saving was 61.20%, with a 5.50% increase in makespan.

In future work, we intend to develop a machine learning-based tool to predict the execution times of the submitted simulation jobs, by using the job scheduler history logs. We also intend to extend the work to include other cloud providers and other simulators. We believe that with those extensions, MSCHEDULER will manage to select more cost-efficient VMs, further reducing financial costs, without sacrificing execution time.

Acknowledgments: This work was funded by Petróleo Brasileiro S.A. – Petrobras, by Project Universal/CNPq n° 404087/2021-3 and CNE/FAPERJ n° E-26/201.012/2022(271103). Also by Spanish Ministry of Science (MICINN, AEI, ERDF/FEDER) under grant agreement PID2021-126248OB-I00, MCIN/AEI/10.13039/ 501100011033/FEDER, UE, and Generalitat de Catalunya (AGAUR) 2021-SGR-00478.

REFERENCES

- [1] M. E. Eldred, A. Orangi, A. A. Al-Emadi, A. A. Ahmad, T. J. O'Reilly, and N. Barghouti, "Reservoir Simulations in a High Performance Cloud Computing Environment," ser. SPE Intelligent Energy International Conference and Exhibition, 2014.
- [2] AWS, "Amazon EC2 Instance Types," <https://aws.amazon.com/ec2/instance-types>, Accessed: May 27th 2023.
- [3] AWS, "Spot Instances: Run fault-tolerant workloads for up to 90% off," <https://aws.amazon.com/ec2/spot>, Accessed: May 27th 2023.
- [4] M. S. Flister and K. Hopstaken, "Running reservoir simulations in the public cloud; a case study of a cost-controlled method, running navigator and eclipse in an azure hpc environment," 2020.
- [5] Z. Noor, Q. Wang, N. Govindaraju, J. Lu, and H. Dong, "Transitioning a Legacy Reservoir Simulator to Cloud Native Services," ser. IPTC, 2020.
- [6] F. A. Portella and F. M. de Souza, "Reservoir simulation in the cloud," in *High Performance Computing in Clouds: Moving HPC Applications to a Scalable and Cost-Effective Environment*. Springer International Publishing, 2023, pp. 265–282.
- [7] C. M. G. Ltd., "Computer Modelling Group - CMG," <https://www.cmg.ca/>, 2023, accessed 20 July 2023.
- [8] L. Teylo, A. L. Nunes, A. C. Melo, C. Boeres, L. M. d. A. Drummond, and N. F. Martins, "Comparing sars-cov-2 sequences using a commercial cloud with a spot instance based dynamic scheduler," in *CCGrid*, 2021, pp. 247–256.
- [9] AWS, "Amazon EC2 Spot Instances Placement Score," <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-placement-score.html>, Accessed: July 3rd 2023.
- [10] AWS, "AWS EC2 Spot Instance Termination Notices," <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html>, Accessed: July 3rd 2023.
- [11] L. Teylo, L. Arantes, P. Sens, and L. M. de A. Drummond, "Scheduling bag-of-tasks in clouds using spot and burstable virtual machines," *IEEE Trans. Cloud Comput.*, 2023.
- [12] R. C. Brum, W. P. Sousa, A. C. M. A. Melo, C. Bentes, M. C. S. de Castro, and L. M. d. A. Drummond, "A fault tolerant and deadline constrained sequence alignment application on cloud-based spot gpu instances," in *Euro-Par*. Springer International Publishing, 2021, pp. 317–333.